

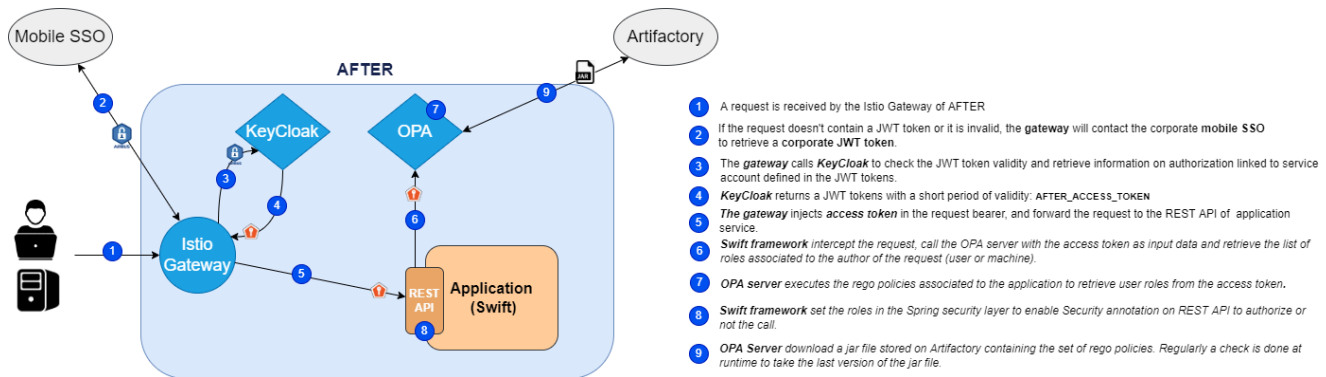
Security

This page explains the main architecture choice to authenticate and authorize requests received by an application implemented with Swift and deployed on AFTER.

- Overview of security workflow
- Authentication
- Authorization

Overview of security workflow

Here is a quick overview of the main flows done during the authentication and authorization process.



To well understand this process, you need to know that:

- Authentication is done by Mobile SSO the first time, but after **Keycloak** has the possibility to generate an **access token** which is send to all applications hosted on AFTER.
- **JWT token** delivered by Mobile SSO is not used in the authorization process, only the **access token** provided by **Keycloak** contains all the authorization attributes of an user (or SA account).
- **Authorizations** defined in **Keycloak** and pushed in the **access token** are used as input for the **OPA policies rules**. Authorization attributes are not linked to application (they are more generic), contrary to roles which are fully defined by applications.
- **Role values** defined on Spring security annotation on REST APIs will be compared to roles returned by the call to **OPA Server**. Each application has its own policies rules, even if all these policies are stored in a same JAR file deployed on **Artifactory**.

[Technical documentation about JWT, and fine grain authorization are provided by AFTER, have a look here.](#)

Authentication

The authentication is supported by the AFTER platform which provides a JWT token in the header of the request.

Here is the default behaviors of Swift:

- By default we consider this token secure and reliable and Swift doesn't check its validity (even if it's possible to enable this feature).
- By default, all APIs requires authentication, that's mean an error is returned if no token is present in the header of the request. Note, that it is possible to exclude some urls.
- By default, Swift configure the Spring Security Context:
 - Retrieve the user name from the JWT token and set it in the UserDetails Spring Object.
 - Store the token in the Thread of the request.

From the application code, it is possible to access to the authentication data.

Here is an example:

```

import java.util.HashMap;
import java.util.Map;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import com.airbus.swift.core.security.auth.service.IAuthenticationService;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;

@Slf4j
@RestController
@RequestMapping("/api")
@RequiredArgsConstructor
public class MyController {

    private final IAuthenticationService authenticationService;

    @GetMapping("/authentication_info")
    public ResponseEntity<Map<String, Object>> getMainStatus() {

        Map<String, Object> info = new HashMap<>();

        info.put("isAuthenticated", authenticationService.isAuthenticated());

        UserDetails userDetails = authenticationService.getPrincipal();
        info.put("userDetails.name", userDetails.getUsername());

        info.put("principalName", authenticationService.getPrincipalName());

        info.put("isAuthenticated", authenticationService.isAuthenticated());

        info.put("jwt", authenticationService.getJWTBody());

        return new ResponseEntity<>(info, HttpStatus.OK);
    }
}

```

and for example a call this API returns the following data:

```

{
  "jwt": {
    "email": "jean.dupont@airbus.com",
    "sub": "8c78b5d6-add2-44ee-84f0-bdc723d8d32d",
    "department": "Subcontractor for IVID",
    "company": "AIRBUS GROUP SA",
    "type": null,
    "attributes": {
      "application": {
        "admin": "user-portal"
      },
      "aircraft": {
        "fomax": false
      },
      "abac": {
        "provisioner": "*"
      }
    },
    "UniqueKey": null,
    "preferred_username": "ng496a2",
    "given_name": "Jean",
    "family_name": "DUPONT",
    "name": "Jean DUPONT",
    "ADAMUniqueKey": null,
    "corp_id": null
  },
  "principalName": "ng496a2",
  "isAuthenticated": true,
  "userDetails.name": "ng496a2"
}

```

Authorization

Like described in the overview chapter, authorization is configured with policies rules defined in OPA.

OPA server is provided by AFTER and Swift send a request to apply the rules to the access token and retrieves a list of roles which are set in the Spring Security context.

Because many steps are done automatically by AFTER platform or Swift framework, at application level , we only need to define:

- **Roles** : Defined the list of roles required to secure the APIs. ([details](#))
- **Add security annotations on REST APIs**: Add Spring security annotation on REST APIs. ([details](#))
- **OPA policies rules**: Implement rego rules to deduct role from user attributes defined in Mobile SSO or Keycloak, package them in a Jar file deployed on Artifactory. ([details](#))